

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Example 5.3 2D Decision boundary for the Iris dataset
5  Machine Learning for Engineering Problem Solving
6  @author: Austin R.J. Downey
7  """
8
9  import IPython as IP
10 IP.get_ipython().run_line_magic('reset', '-sf')
11
12
13 import numpy as np
14 import matplotlib.pyplot as plt
15 import sklearn as sk
16
17
18 cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
19 plt.close('all')
20
21
22 %% Load your data
23
24 # We will use the Iris data set. This dataset was created by biologist Ronald
25 # Fisher in his 1936 paper "The use of multiple measurements in taxonomic
26 # problems" as an example of linear discriminant analysis
27
28 iris = sk.datasets.load_iris()
29
30 # for simplicity, extract some of the data sets
31 X = iris['data'] # this contains the length of the petals and sepals
32 Y = iris['target'] # contains what type of flower it is
33 Y_names = iris['target_names'] # contains the name that aligns with the type of the
34 # flower
35 feature_names = iris['feature_names'] # the names of the features
36
37 # plot the Sepal data
38 plt.figure(figsize=(6.5,3))
39 plt.subplot(121)
40 plt.grid(True)
41 plt.scatter(X[Y==0,0],X[Y==0,1],marker='o',zorder=10)
42 plt.scatter(X[Y==1,0],X[Y==1,1],marker='s',zorder=10)
43 plt.scatter(X[Y==2,0],X[Y==2,1],marker='d',zorder=10)
44 plt.xlabel(feature_names[0])
45 plt.ylabel(feature_names[1])
46
47 plt.subplot(122)
48 plt.grid(True)
49 plt.scatter(X[Y==0,2],X[Y==0,3],marker='o',label=Y_names[0],zorder=10)
50 plt.scatter(X[Y==1,2],X[Y==1,3],marker='s',label=Y_names[1],zorder=10)
51 plt.scatter(X[Y==2,2],X[Y==2,3],marker='d',label=Y_names[2],zorder=10)
52 plt.xlabel(feature_names[2])
53 plt.ylabel(feature_names[3])
54 plt.legend(framealpha=1)
55 plt.tight_layout()
56
57
58 %% plot the Linear decision boundary in 2D "Petal" space
59
60 # build the training and target set.
61 X_train = X[:, (2, 3)] # petal length, petal width
62 y_train = Y == 2
63
64 # build the Logistic Regression model
65 log_reg = sk.linear_model.LogisticRegression(C=10**10)
66 # Note: The hyper-parameter controlling the regularization strength of a Scikit-Learn

```

```

67 # LogisticRegression model is not alpha (as in other linear models), but its
68 # inverse: C. The higher the value of C, the less the model is regularized.
69
70 # train the Logistic Regression model
71 log_reg.fit(X_train, y_train)
72
73 # build the x values for the predictions over the entire "petal space"
74 x_grid, y_grid = np.meshgrid(
75     np.linspace(2.8, 7, 500),
76     np.linspace(0.3, 3, 200),
77 )
78 X_new = np.vstack((x_grid.reshape(-1), y_grid.reshape(-1))).T # build a vector format of
the mesh grid
79
80 # predict on the vectorized format
81 y_predict = log_reg.predict(X_new)
82 y_proba = log_reg.predict_proba(X_new)
83
84 # convert back to meshgrid shape for plotting
85 zz_predict = y_predict.reshape(x_grid.shape)
86 zz_proba = y_proba[:, 1].reshape(x_grid.shape)
87
88 # plot the 2D "petal space"
89 plt.figure(figsize=(6.5,3))
90 plt.grid(True)
91 plt.scatter(X[Y==1,2],X[Y==1,3],marker='s',color=cc[1],label=Y_names[1],zorder=10)
92 plt.scatter(X[Y==2,2],X[Y==2,3],marker='d',color=cc[2],label=Y_names[2],zorder=10)
93 plt.contourf(x_grid, y_grid, zz_predict, cmap='Pastel2')
94 contour = plt.contour(x_grid, y_grid, zz_proba, [0.100,0.5,0.900],cmap=plt.cm.brg)
95 plt.clabel(contour, inline=1) # add the labels to the plot
96 plt.xlabel(feature_names[2])
97 plt.ylabel(feature_names[3])
98 plt.legend()
99 plt.tight_layout()
100
101
102

```